INFORMATION MANAGEMENT SYSTEM

LAB ASSIGNMENT - 06

(2503A52060-B2-T.Sowmya)

1 Problem Statement

Designacompletelibrarydatabase.

2 Tasks

Model Book, Author, Member, Librarian, Issue, Return and Fine. Define relationships.

Normalize.

Implement constraints.

Implement overdue/finequeries.

3 Expected Deliverables

Requirement Analysis

Entities: Book, Author, Member, Librarian, Issue, Return, Fine

Functional requirements

- Store details of books available in the library.
- Store details of authors and their books.
- Store member information.
- Store librarian information.
- Record when a book is issued to a member.
- Record when an issued book is returned.
- Calculate and store fines for overdue books.
- Identify books that are currently overdue.
- Display the fine amount for members.
- Maintain data consistency using primary keys and foreign key

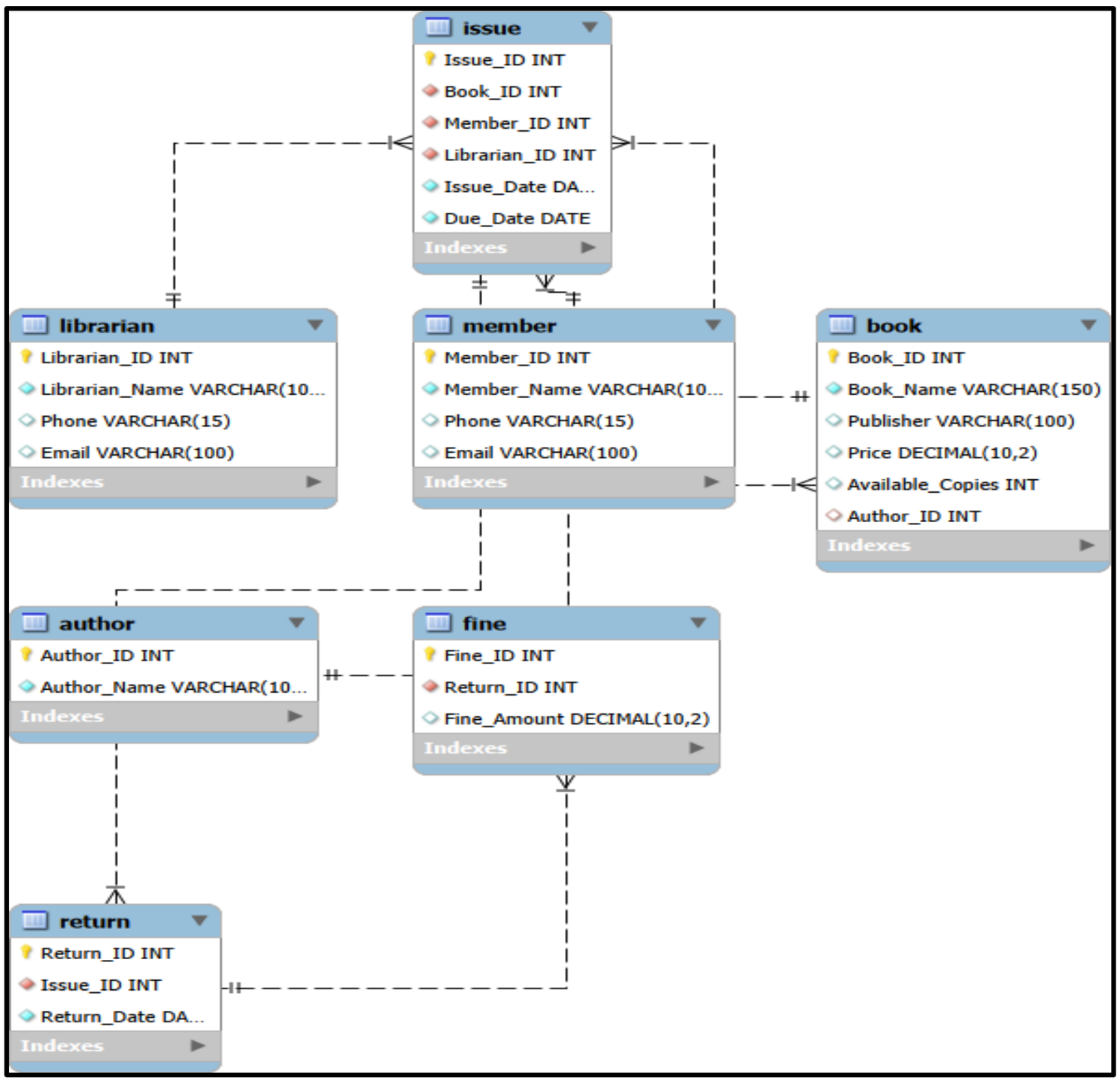
ERD

Relationships and Cardinality

- Author – Book (M : N): One author can write many books, and one book can be written by many authors.
- Member – Issue (1 : N): One member can have many issue records over time, but each issue belongs to one member.
- Librarian – Issue (1 : N): One l ibrarian can process many issue records, but each issue is processed by one librarian.
- Book – Issue (1 : N): One book can be issued many times over time, but each issue record refers to one book.

- Issue – Return (1 : 1): One issue can have one return record, and each return record belongs to one issue.
- Return – Fine (1 : 0..1): One return can have zero or one fine. A fine is created only when the book is returned late.

Entity- Relationship Diagram:



SQL:

```
CREATE DATABASE LibraryDB;
USE LibraryDB; CREATE
TABLE Author (
   Author_I D INT PRIMARY KEY, Author_Name
   VARCHAR(1 00) NOT NULL
);
CREATE TABLE Book ( Book_I
   D INT PRIMARY KEY,
   Book_Name VARCHAR(1 50) NOT NULL,
```

```
    Publisher VARCHAR(1 00),
    Price DECIMAL(1 0,2) CHECK (Price >= 0),
    Available_Copies INT CHECK (Available_Copies >= 0),
    Author_ID INT,
    FOREIGN KEY (Author_ID) REFERENCES Author(Author_ID)
);
```

```
CREATE TABLE Member (
   Member_ID INT PRIMARY KEY,
   Member_Name VARCHAR(1 00) NOT
   NULL, Phone VARCHAR(1 5) UNIQUE,
   Email VARCHAR(1 00) UNI QUE
);
CREATE TABLE Librarian (
   Librarian_I D INT PRIMARY KEY,
   Librarian_Name VARCHAR(1 00) NOT NULL,
   Phone VARCHAR(1 5),
   Email VARCHAR(1 00) UNI QUE
);
CREATE TABLE Issue ( I
   ssue_ID INT PRIMARY
   KEY, Book_ID
   INT NOT NULL,
   Member_I D INT NOT
   NULL, Librarian_ID INT
   NOT NULL, Issue_Date
   DATE NOT NULL,
   Due_Date DATE NOT
   NULL,
   FOREIGN KEY (Book_ID) REFERENCES Book(Book_ID),
   FOREIGN KEY (Member_ID) REFERENCES Member(Member_ID),
   FOREIGN KEY (Librarian_ID) REFERENCES Librarian(Librarian_I
   D), CHECK (Due_Date
   >= Issue_Date)
);
CREATE TABLE `Return` (
   Return_I D INT PRIMARY KEY,
   Issue_I D INT UNIQUE NOT
   NULL,
```

```
    Return_Date DATE NOT NULL,
    FOREIGN KEY (Issue_ID) REFERENCES Issue(I ssue_ID)
);
CREATE TABLE Fine (
    Fine_I D INT PRIMARY
    KEY,
    Return_ID INT UNIQUE NOT NULL,
    Fine_Amount DECIMAL(1 0,2) CHECK (Fine_Amount >=
    0), FOREIGN KEY (Return_ID) REFERENCES
    `Return`(Return_ID)
);
INSERT I NTO Author VALUES
(1 , 'R.K. Narayan'),
(2, 'Chetan Bhagat'),
(3, 'J. K. Rowling'),
(4, 'George Orwell');
INSERT I NTO Book VALUES
(1 01 , 'Malgudi Days', 'Indian Publisher', 350.00, 5, 1 ),
(1 02, 'Five Point Someone', 'Rupa', 299.00, 4, 2),
(1 03, 'Harry Potter', 'Bloomsbury', 550.00, 3, 3),
```

(1 04, '1 984', 'Penguin', 450.00, 2, 4);

INSERT I NTO Member VALUES

(201 , 'Siri', '987654321 0', 'siri@gmail.com'),

(202, 'Anu', '987654321 1 ', 'anu@gmail.com'),

```
(203, 'Rahul', '987654321 2', 'rahul@gmail.com');
INSERT I NTO Librarian VALUES
(301 , 'Ramesh', '9000000001 ', 'ramesh@library.com'),
(302, 'Priya', '9000000002', 'priya@library.com');
INSERT I NTO Issue VALUES
(401 , 1 01 , 201 , 301 , '2026-09-01 ', '2026-09-07'),
(402, 1 02, 202, 302, '2026-09-05', '2026-09-1 2'),
(403, 1 03, 203, 301 , '2026-09-1 0', '2026-09-1 7');
INSERT I NTO `Return` VALUES
(501 , 401 , '2026-09-06'),
(502, 402, '2026-09-1 5'); INSERT
INTO Fine VALUES (601 ,
501 , 0.00),
(602, 502, 1 5.00);

SELECT
    B.Book_I D,
    B.Book_Name,
    A.Author_Name,
    B.Publisher, B.Price,
    B.Available_Copies
FROM Book B
JOI N Author A ON B.Author_I D = A.Author_ID;

SELECT
    I. Issue_ID,
    B.Book_Name,
    M. Member_Name,
    I. Issue_Date,
    I. Due_Date
FROM I ssue I
JOI N Book B ON I.Book_ID = B. Book_ID
```

```
JOIN Member M ON I.Member_ID = M.Member_ID;

SELECT
    I.Issue_ID,
    B.Book_Name,
    M.Member_Name,
    I.Due_Date
FROM Issue I
JOIN Book B ON I.Book_ID = B.Book_ID
JOIN Member M ON I.Member_ID =
M.Member_ID LEFT JOIN `Return` R ON I.I
ssue_ID = R.Issue_ID
WHERE R.Return_ID IS NULL AND I.Due_Date < CURDATE();
```

```
SELECT
    I. Issue_ID,
    B.Book_Name,
```

```
    M. Member_Name,
    I. Due_Date,
    DATEDI FF(CURDATE(),          I.Due_Date)   AS Overdue_Days,
    DATEDI FF(CURDATE(), I.Due_Date) * 5 AS
Fine_Amount FROM I ssue I
JOI N Book B ON I.Book_ID = B. Book_ID
JOI N Member M ON I.Member_ID =
M.Member_ID LEFT JOIN `Return` R ON
I.Issue_ID = R.Issue_ID WHERE R.Return_ID I
S NULL
AND I .Due_Date < CURDATE();

SELECT
    F.Fine_ID,
    M. Member_Name,
    B.Book_Name,
    F.Fine_Amount
FROM Fine F
JOI N `Return` R ON F.Return_ID = R.Return_ID JOIN
Issue I ON R.Issue_ID = I .Issue_ID
JOI N Member M ON I .Member_I D =
M.Member_ID JOI N Book B ON I.Book_ID = B.
Book_ID;
```

OUTPUTS:

DISPLAY BOOK DETAILS WITH AUTHORS

SQL File 1* SQL File 3* SQL File 4* SQL File 5* SQL File 6*

Limit to 1000 rows

```
78      (602, 502, 15.00);
79
80      SELECT
81          B.Book_ID,
82          B.Book_Name,
83          A.Author_Name,
84          B.Publisher,
85          B.Price,
86          B.Available_Copies
87      FROM Book B
88      JOIN Author A
89      ON B.Author_ID = A.Author_ID;
90
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Book_ID	Book_Name	Author_Name	Publisher	Price	Available_Copies
101	Malgudi Days	R.K. Narayan	Indian Publisher	350.00	5
102	Five Point Someone	Chetan Bhagat	Rupa	299.00	4
103	Harry Potter	J.K. Rowling	Bloomsbury	550.00	3
104	1984	George Orwell	Penguin	450.00	2

DISPLAY ISSUE DETAILS

SQL File 1* SQL File 3* SQL File 4* SQL File 5* SQL File 6*

Limit to 1000 rows

```
90
91  SELECT
92      I.Issue_ID,
93      B.Book_Name,
94      M.Member_Name,
95      I.Issue_Date,
96      I.Due_Date
97  FROM Issue I
98  JOIN Book B
99  ON I.Book_ID = B.Book_ID
100 JOIN Member M
101 ON I.Member_ID = M.Member_ID;
102
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Issue_ID	Book_Name	Member_Name	Issue_Date	Due_Date
401	Malgudi Days	Siri	2026-09-01	2026-09-07
402	Five Point Someone	Anu	2026-09-05	2026-09-12
403	Harry Potter	Rahul	2026-09-10	2026-09-17

DI SPLAY OVERDUE BOOKS

SQL File 1* SQL File 3* SQL File 4* SQL File 5* SQL File 6*

Limit to 1000 rows

```
102
103 SELECT
104     I.Issue_ID,
105     B.Book_Name,
106     M.Member_Name,
107     I.Due_Date
108 FROM Issue I
109 JOIN Book B
110 ON I.Book_ID = B.Book_ID
111 JOIN Member M
112 ON I.Member_ID = M.Member_ID
113 LEFT JOIN `Return` R
114 ON I.Issue_ID = R.Issue_ID
115 WHERE R.Return_ID IS NULL
116 AND I.Due_Date < CURDATE();
117
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Issue_ID	Book_Name	Member_Name	Due_Date
403	Harry Potter	Rahul	2026-09-17

CALCULATEFINEFOROVERDUEBOOKS

SQL File 1* | SQL File 3* | SQL File 4* | SQL File 5* | SQL File 6*

Limit to 1000 rows

```
117
118 •  SELECT
119        I.Issue_ID,
120        B.Book_Name,
121        M.Member_Name,
122        I.Due_Date,
123        DATEDIFF(CURDATE(), I.Due_Date) AS Overdue_Days,
124        DATEDIFF(CURDATE(), I.Due_Date) * 5 AS Fine_Amount
125    FROM Issue I
126    JOIN Book B
127    ON I.Book_ID = B.Book_ID
128    JOIN Member M
129    ON I.Member_ID = M.Member_ID
130    LEFT JOIN `Return` R
131    ON I.Issue_ID = R.Issue_ID
132    WHERE R.Return_ID IS NULL
133    AND I.Due_Date < CURDATE();
134
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Issue_ID	Book_Name	Member_Name	Due_Date	Overdue_Days	Fine_Amount
403	Harry Potter	Rahul	2026-09-17	2	10

DISPLAYSTOREDFINES

```
134
135 •  SELECT
136        F.Fine_ID,
137        M.Member_Name,
138        B.Book_Name,
139        F.Fine_Amount
140    FROM Fine F
141    JOIN `Return` R
142    ON F.Return_ID = R.Return_ID
143    JOIN Issue I
144    ON R.Issue_ID = I.Issue_ID
145    JOIN Member M
146    ON I.Member_ID = M.Member_ID
147    JOIN Book B
148    ON I.Book_ID = B.Book_ID;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Fine_ID	Member_Name	Book_Name	Fine_Amount
601	Siri	Malgudi Days	0.00
602	Anu	Five Point Someone	15.00

Constraints Implemented

- PRIMARY KEY: Uniquely identifies each record in a table.
- FOREIGN KEY: Maintains relationships between related tables.

- NOT NULL: Ensures required fields cannot be left empty.

- UNIQUE: Prevents duplicate values such as phone numbers and email addresses.
- CHECK: Ensures valid values, such as non-negative price, available copies, and fine

  amount.
- DEFAULT: Provides a default value when no value is specified.
- Referential Integrity: Ensures that foreign key values match existing records in the

  related table.

4 Viva Questions

1 How do you model Issue and Return?

Issueand Return are modeled asseparatetables. TheIssuetablestoresbookissuedetails such as issuedate and due date. TheReturntablestores the returndateand islinked toIssueusing `Issue_ID`.

2 What is a Foreign Key?

AForeign Keyisa columnthat refers totheprimarykeyofanothertable. Itisusedtoestablish relationshipsbetween tablesand maintain referential integrity.

3 Why is Author kept as a separate table?

Authoris keptas aseparatetable toavoid duplicateauthorinformation and reducedataredundancy. Italsomakesthedatabaseeasiertomaintainand supportsthemany-to-manyrelationshipbetween Authorand Book.